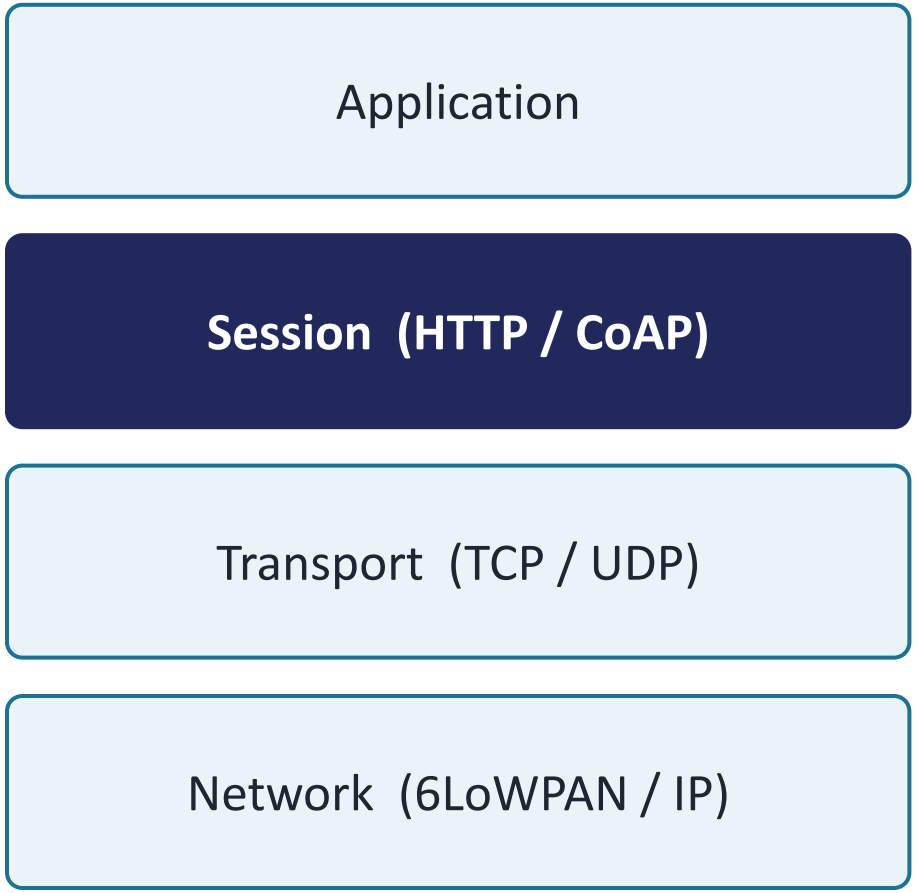
Unit-4
Transport and Session Layer Protocols

- **Transport Layer**
  - TCP
  - MPTCP
  - UDP
  - DCCP
  - SCTP
  - TLS
  - DTLS

- **Session Layer**
  - HTTP
  - CoAP
  - XMPP
  - AMQP
  - MQTT

The Session Layer in IoT

- Governs how two devices open, manage and use a data exchange session between applications.
- In IoT this layer must work under tight constraints: low power, small memory, and lossy, low-bandwidth links.
- Two dominant choices sit here: HTTP, borrowed from the web, and CoAP, purpose-built for constrained devices.
- Choice of protocol shapes battery life, latency and how well a device fits into existing web infrastructure.

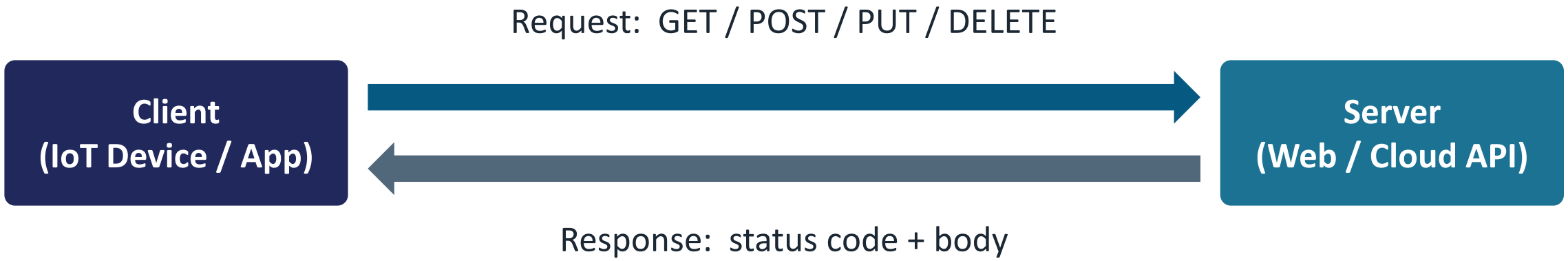


HTTP in IoT — Overview



- HyperText Transfer Protocol — the same protocol that powers the web, running over TCP.
- In IoT it fits gateways, edge servers and cloud dashboards: devices with enough power and memory to spare.
- Follows a stateless, client-server, request-response model.
- Text-based (plain headers, often JSON payloads) and backed by mature libraries, proxies and security tooling already deployed across the internet.

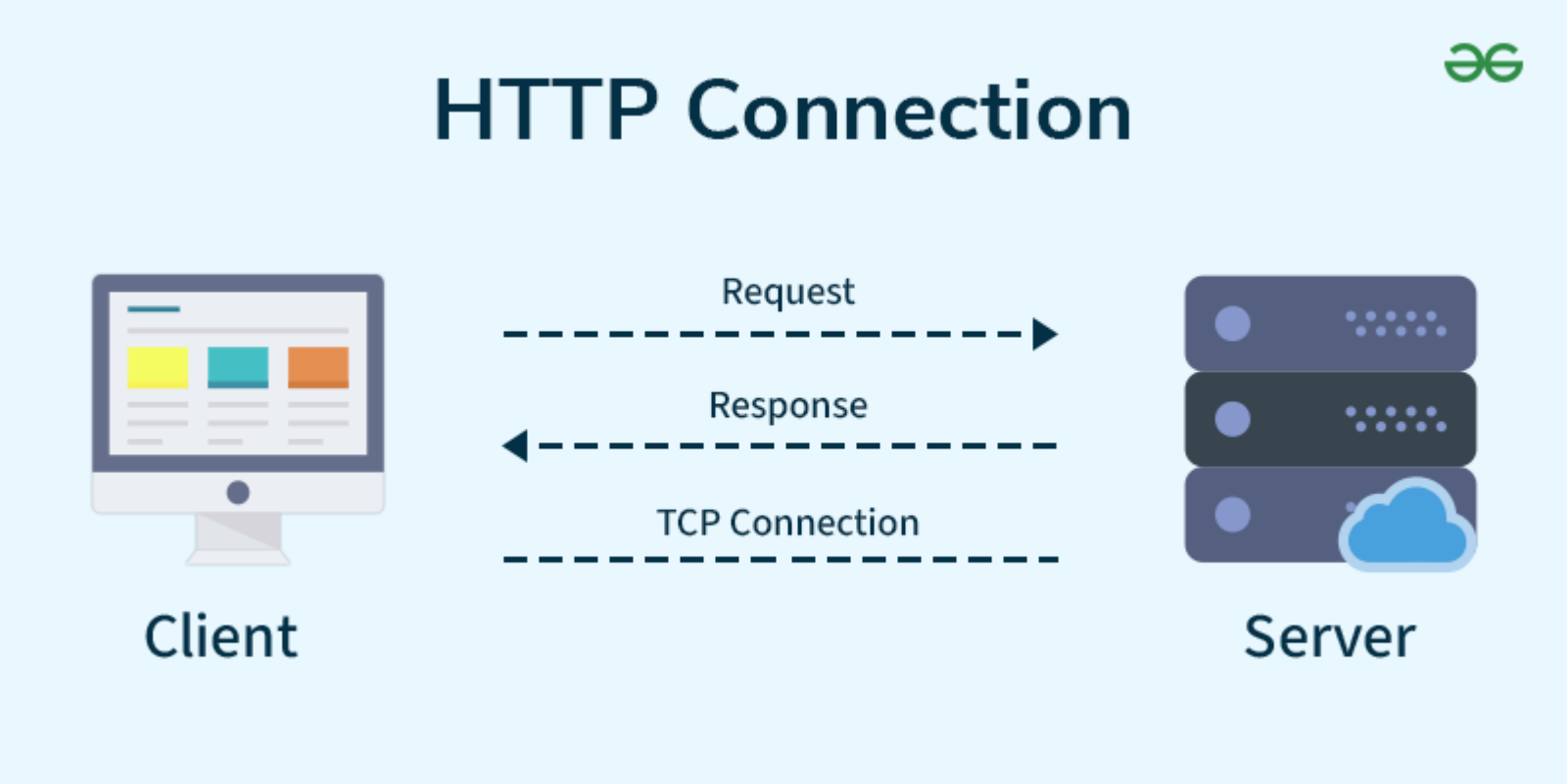
HTTP — Request / Response Model



- GET reads a resource; POST creates or submits data; PUT replaces a resource; DELETE removes it.
- Every response carries a status code, e.g. 200 OK, 201 Created, 404 Not Found, 500 Server Error.
- Each request opens a new logical exchange — HTTP itself keeps no memory of prior calls (stateless).

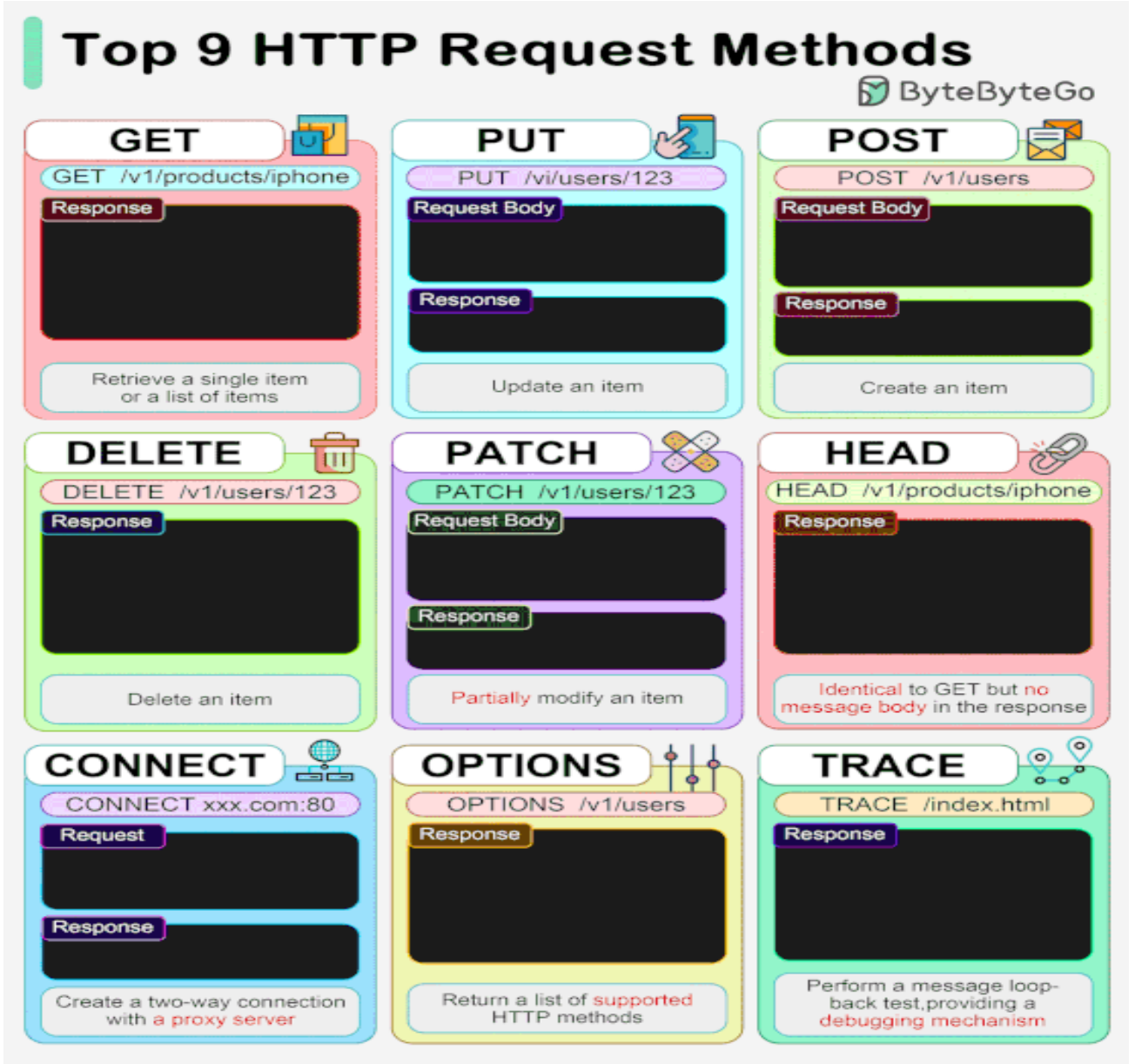
HTTP

Client-Server Model & Request-Response Cycle :



HTTP

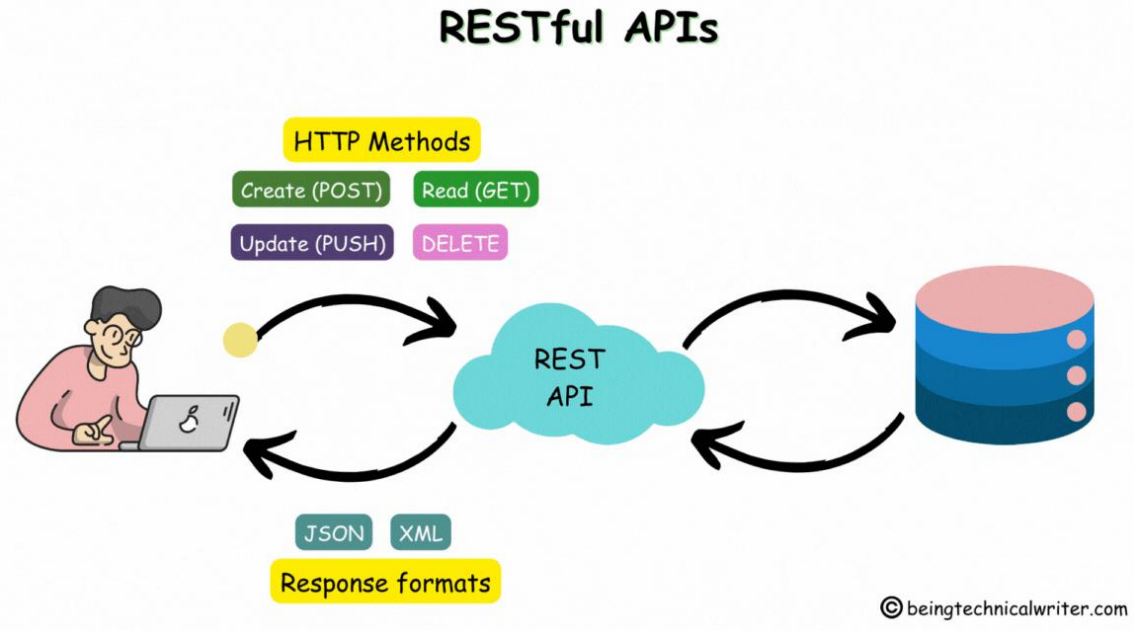
HTTP Methods



HTTP

4. RESTful APIs

- A RESTful API, or a REST API, is a type of web service that conforms to the principles of REST (Representational State Transfer).
- REST is an architectural style for designing networked applications that are simple, scalable, and reliable.



HTTP

How RESTful APIs Use HTTP?

HTTP Methods: RESTful APIs utilize HTTP methods to represent different actions on resources:

- GET: Retrieve a resource.
- POST: Create a new resource.
- PUT: Update an existing resource.
- DELETE: Delete a resource.
- PATCH: Apply partial modifications to a resource.
- URLs: Resources are identified by their URLs, which typically follow a hierarchical structure.
- Status Codes: HTTP status codes are used to indicate the success or failure of a request.
- Data Formats: REST APIs commonly use JSON or XML to represent data in a structured format.

HTTP — Limitations for Constrained IoT

1 Heavy Headers

Verbose, text-based headers waste bytes on links that may carry only tens of bytes per packet.

2 TCP Overhead

Connection setup (3-way handshake) and teardown cost time and energy on every exchange.

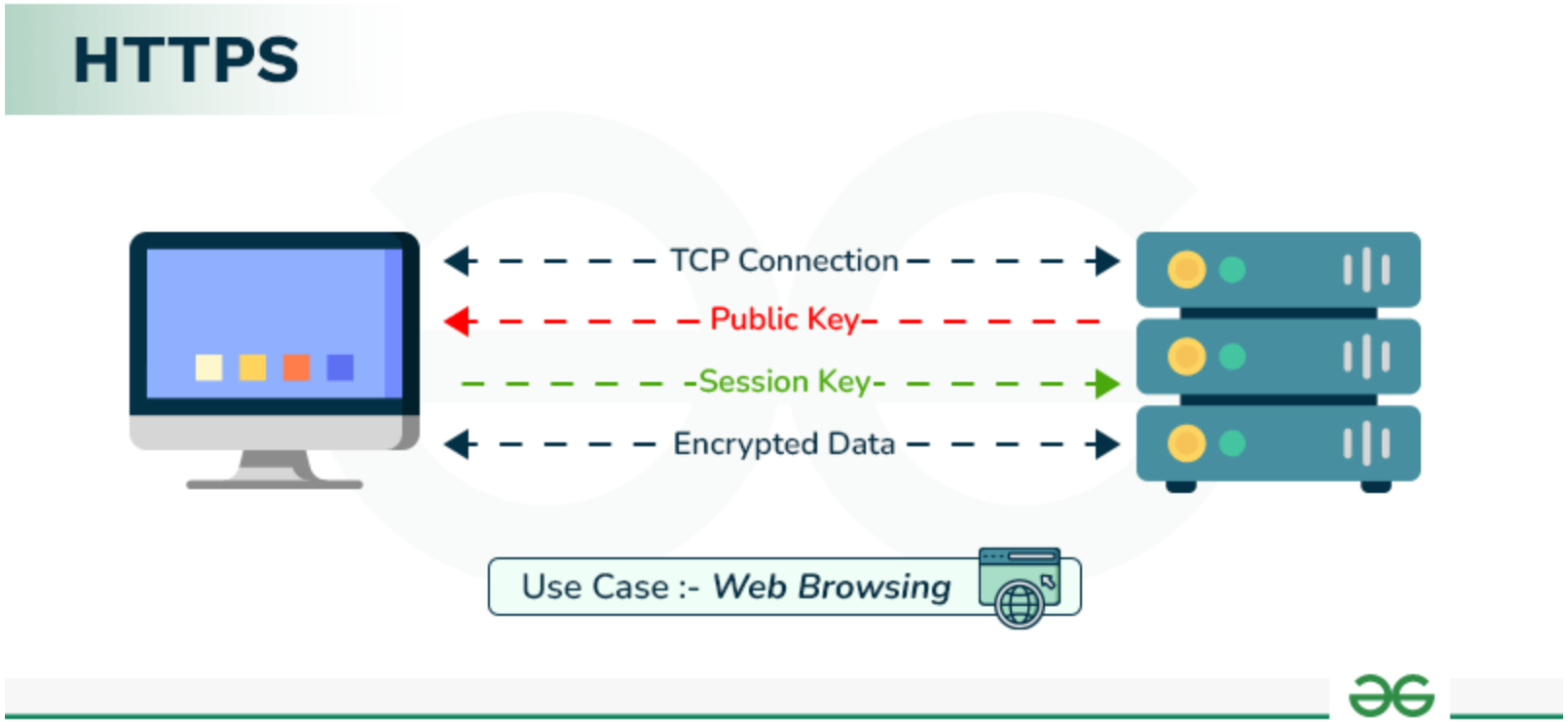
3 Always-On Model

Designed around always-connected servers, not battery devices that sleep for most of their life.

4 No Native Push

No built-in support for multicast or subscribing to resource changes (observe).

HTTP


HTTPS
TCP Connection
Public Key
-Session Key-
Encrypted Data
Use Case :- Web Browsing

CoAP — Built for Constrained Devices



- Constrained Application Protocol — specified in RFC 7252 for low-power, low-memory nodes and lossy networks.
- Runs over UDP instead of TCP, avoiding connection-setup overhead.
- REST-like: reuses the same GET / POST / PUT / DELETE model as HTTP, so it maps naturally onto web APIs.
- Uses a compact binary header (as small as 4 bytes) instead of verbose text headers.

CoAP — Two-Layer Architecture

Application	Sensors, actuators and IoT services using CoAP resources
Request / Response Layer	REST semantics — GET, POST, PUT, DELETE on resources (URIs)
Messaging Layer	Adds reliability, deduplication and ordering on top of UDP
UDP / Transport	Lightweight, connectionless datagram delivery

CoAP — Compact Message Format

Ver / T / TKL

Version, message Type, Token Length — packed into the first byte

Code

Method (GET/POST/PUT/DELETE) or response status

Message ID

Matches requests to responses and detects duplicates

Token

Correlates a response to its request

Options

URI path, content format and other metadata

Payload

The actual resource data, marked off by an 0xFF byte

CoAP — Reliability over UDP

UDP has no built-in reliability, so CoAP's messaging layer defines four message types to add it back where needed:

Type	Meaning	Role
CON	Confirmable	Must be acknowledged by the receiver; retransmitted if no ACK arrives
NON	Non-confirmable	Fire-and-forget — no acknowledgement expected (e.g. repeated sensor readings)
ACK	Acknowledgement	Confirms receipt of a CON message, optionally carrying the response
RST	Reset	Signals that a message could not be processed

CoAP — Beyond Basic Request/Response

1 Observe

A client subscribes to a resource and receives updates whenever it changes — near real-time without polling.

2 Block-wise Transfer

Splits large payloads into smaller blocks that fit constrained link budgets and memory.

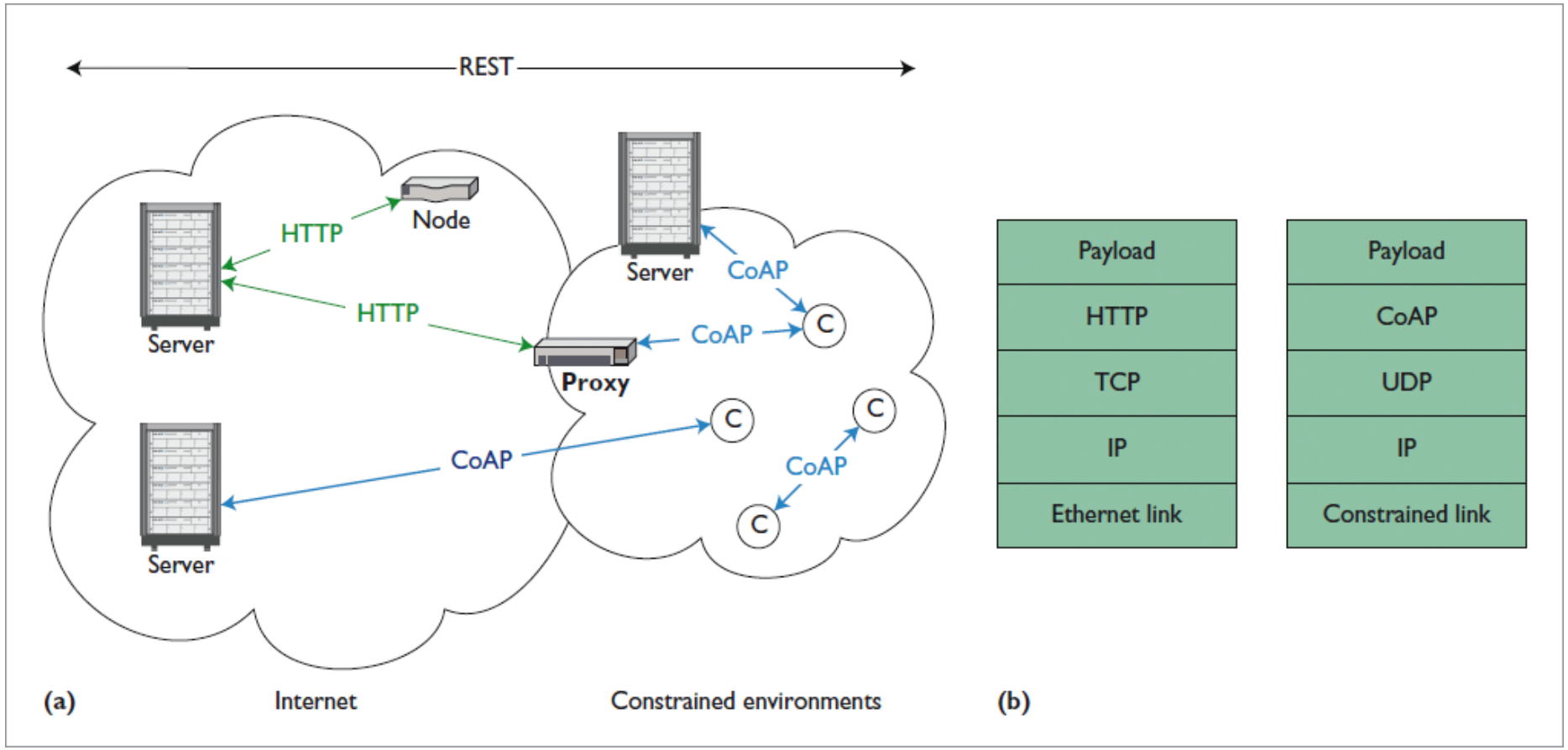
3 DTLS Security

Datagram TLS secures CoAP over UDP, mirroring HTTPS for the constrained world.

4 HTTP–CoAP Proxying

Cross-protocol proxies let CoAP devices interoperate with ordinary HTTP web services.

CoAP-Archtecture



SUMMARY

HTTP vs. CoAP — Choosing the Right Fit

Aspect	HTTP	CoAP
Transport	TCP	UDP
Header size	Verbose, text-based	Compact, binary (4 bytes+)
Model	Request / response only	Request/response + Observe
Power fit	Needs steady power	Built for battery devices
Best suited for	Gateways, cloud, dashboards	Sensors, actuators, edge nodes

Takeaway: use HTTP where power and bandwidth are plentiful; use CoAP where devices are constrained — the two can interoperate through a proxy.

Session Layer Protocols- MQTT, XMPP, AMQP

M

MQTT

Lightweight publish/subscribe messaging for constrained devices

X

XMPP

XML-based messaging with real-time presence

A

AMQP

Enterprise-grade, reliable message queuing

MQTT — Publish / Subscribe Architecture



Publishers send data on a topic; the Broker routes it to every Subscriber of that topic — publishers and subscribers never connect directly.

- Broker: central server that filters and routes messages by topic name
- Client: any device acting as publisher, subscriber, or both
- Topic: hierarchical string (e.g. home/livingroom/temp) used for routing
- Runs over TCP/IP with a minimal 2-byte fixed header — very low overhead
- Designed for low-bandwidth, high-latency or unreliable networks

MQTT — Quality of Service (QoS) Levels

QoS 0

At Most Once

Fire-and-forget delivery. No acknowledgment or retry — fastest, least reliable.

QoS 1

At Least Once

Message is acknowledged (PUBACK). May be delivered more than once.

QoS 2

Exactly Once

Four-way handshake guarantees single, assured delivery. Highest overhead.

MQTT — Key Features & IoT Use Cases

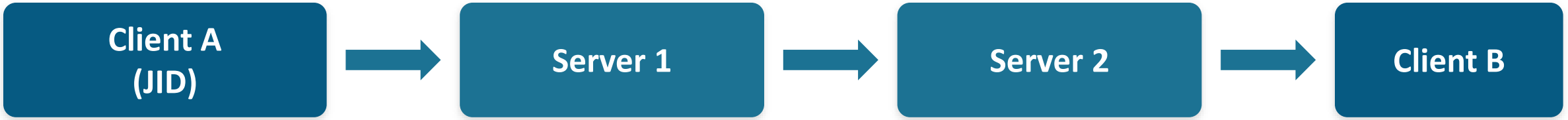
Key Features

- 2-byte fixed header — minimal protocol overhead
- Retained messages: last known value delivered to new subscribers
- Last Will & Testament: auto-notifies on ungraceful disconnect
- Very low bandwidth and power consumption

IoT Use Cases

- Smart home automation
- Remote environmental sensors
- Fleet & vehicle telematics
- Healthcare wearables

XMPP — Extensible Messaging & Presence Protocol



Federated network: servers relay XML stanzas between clients registered on different domains — similar in spirit to email routing.

- Based on XML stanzas (<message/>, <presence/>, <iq/>) over persistent TCP connections
- Every entity has a unique Jabber ID (JID), e.g. sensor1@building.example.com
- Decentralized and federated — no single central broker, like email servers
- Extended for IoT through XEPs (XMPP Extension Protocols), e.g. XEP-0323 for sensor data

XMPP — Key Features & IoT Use Cases

Key Features

- Real-time presence detection (online / offline / busy)
- Highly extensible through XEP add-on specifications
- Federated, decentralized client-server topology
- Built-in security via TLS encryption and SASL authentication

IoT Use Cases

- Smart appliance status updates
- Chat-based device control
- Social IoT & collaborative sensing
- Presence-aware energy management

AMQP — Advanced Message Queuing Protocol



The Exchange routes messages into one or more Queues using Bindings; Consumers pull messages from their Queue.

- Binary, platform-independent wire-level protocol (not just an API)
- Exchange: receives messages and routes them via bindings and rules
- Queue: stores messages durably until a consumer retrieves them
- Guarantees reliable, ordered delivery — supports transactions and acknowledgments

AMQP — Key Features & IoT Use Cases

Key Features

- Reliable, transactional messaging with delivery guarantees
- Flow control and explicit message acknowledgment
- Fully interoperable across vendors and platforms
- Supports complex routing: direct, fanout, topic, headers

IoT Use Cases

- Industrial IoT & SCADA systems
- Financial-grade sensor pipelines
- Cloud-to-cloud integration
- Smart grid communication

COMPARISON

MQTT vs XMPP vs AMQP

Parameter	MQTT	XMPP	AMQP
Message Format	Binary	XML	Binary
Model	Publish / Subscribe	Client-Server (federated)	Queue-based routing
Overhead	Very Low	High	Medium
Reliability	3 QoS levels	Best-effort	Transactions + ACKs
Best For	Constrained devices	Presence & social IoT	Enterprise messaging

Unit-4
Transport and Session Layer Protocols

XMPP

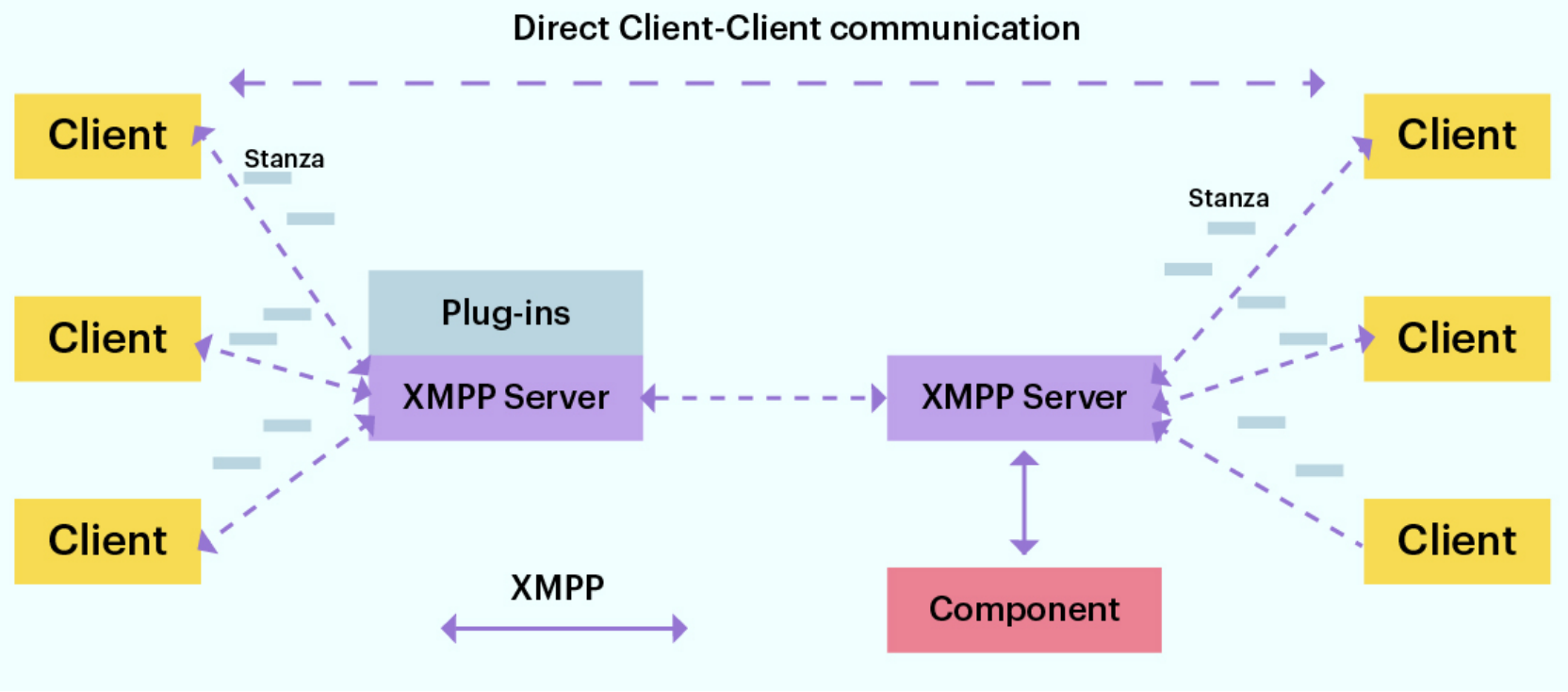
XMPP can handle many of the core functions of a chat app including:

- Sending and receiving messages
- Broadcasting a user's status
- Managing a friends/contact list
- Blocking users from sending you messages
- The History of XMPP

Unit-4
Transport and Session Layer Protocols

XMPP

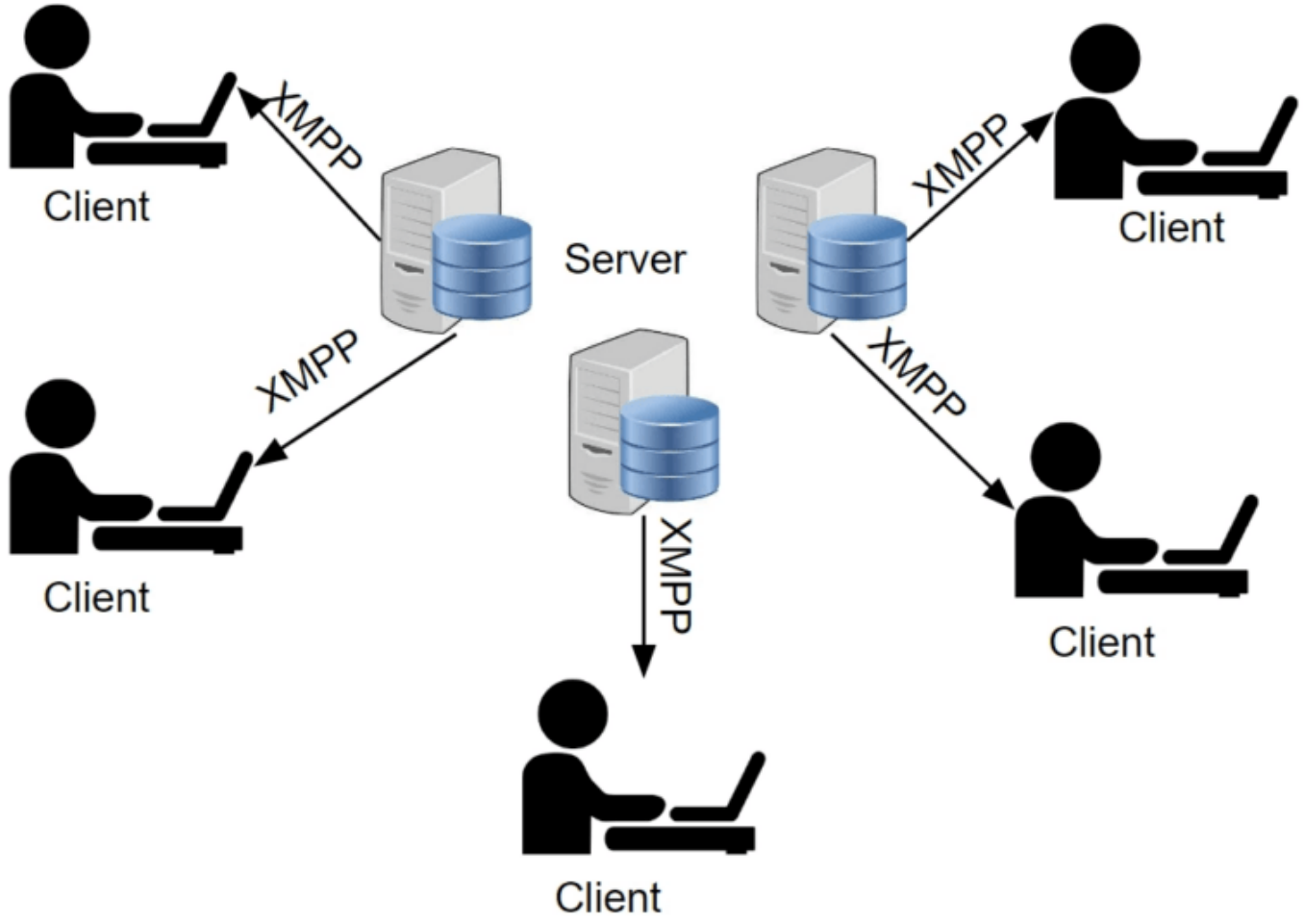
XMPP architecture



Unit-4
Transport and Session Layer Protocols

XMPP

XMPP architecture



Unit-4
Transport and Session Layer Protocols

XMPP-Working

- XMPP works on a client-server architecture.
- This means that when you send a message via XMPP, it's first sent to a server which then routes it to the correct client (user).
- How does it know which client to send your message to?
  - Well, each client connected to an XMPP server is assigned a unique identifier, known as a Jabber ID.
  - The current format of the unique identifier is user@domain.com/resource.

Unit-4
Transport and Session Layer Protocols

XMPP-Working

How XMPP Works:

1.Client-Server Model: Users connect to an XMPP server using a client application (e.g., a chat client).
2.Message Exchange: Messages are exchanged between users through the XMPP server.
3.Presence Information: Users can publish their presence information to the server, which is then shared with their contacts.
4.Extensibility: XMPP's XML-based structure allows for the creation of custom extensions for various applications, such as file transfer, voice/video calls, and more.

Unit-4
Transport and Session Layer Protocols

XMPP

XMPP Specifications & Standards

- The IETF published the core XMPP standards in 2004 in RFC 3920 and RFC 3921.
- Then, in 2011, the IETF updated the core definition of XMPP in RFC 6120 and RFC 6121.
- In addition to updating the core, IETF published a few more specifications over the years:
- RFC 7395, which defines the use of XMPP over WebSockets via a binding;
- RFC 7590, which allows for the use of TLS (Transport Layer Security); and
- RFC 7622, which modernized the XMPP addresses.

Unit-4
Transport and Session Layer Protocols

XMPP

Core Concepts:

- XML-based: XMPP uses XML (Extensible Markup Language) to structure and exchange data, allowing for flexibility and extensibility.
- Decentralized: Unlike many proprietary messaging services, XMPP doesn't rely on a single central server. Users can connect to any XMPP server they choose, promoting interoperability and user control.
- Presence Information: XMPP allows users to share their availability status (online, away, busy, etc.) with their contacts.
- Extensible: The protocol can be extended with custom features and applications using XML extensions.

Unit-4
Transport and Session Layer Protocols

XMPP

XMPP Features:

- **Instant Messaging:** Send and receive text messages in real-time.
- **Group Chat:** Participate in group conversations with multiple users.
- **File Transfer:** Share files with other users.
- **Voice and Video Calls:** Make voice and video calls over IP (VoIP).
- **Presence Indication:** Share your availability status with others.
- **Contact Management:** Manage your contact list and receive notifications when contacts come online or offline.

Unit-4
Transport and Session Layer Protocols

XMPP

Advantages of XMPP:

- Open Source: XMPP is an open standard, promoting interoperability and innovation.
- Decentralized: Users have more control over their communication and data.
- Flexible: The protocol can be adapted to various use cases beyond basic messaging.
- Secure: XMPP supports encryption to protect user communication.

Unit-4
Transport and Session Layer Protocols

XMPP

Disadvantages of XMPP:

- Smaller User Base: Compared to proprietary messaging services like WhatsApp or Messenger, XMPP has a smaller user base.

- Steeper Learning Curve: Setting up and configuring an XMPP server can have a steeper learning curve than using a proprietary service.

Unit-4
Transport and Session Layer Protocols

XMPP

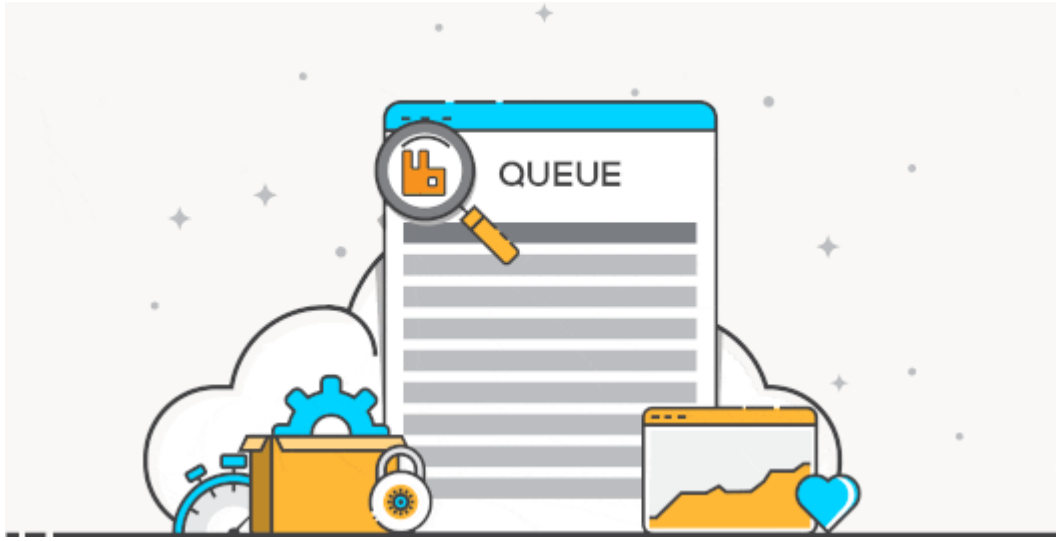
Applications of XMPP:

- Instant Messaging Clients: Popular XMPP clients include Pidgin, Gajim, and Converse.js.
- Enterprise Communication: XMPP can be used for internal communication within organizations.
- IoT (Internet of Things): XMPP can be used for machine-to-machine communication in IoT applications.
- Presence-based Applications: XMPP can be used to build applications that rely on real-time presence information, such as online games or collaborative tools.

Unit-4
Transport and Session Layer Protocols

AMQP

AMQP (Advanced Message Queuing Protocol) is an open standard application layer protocol for message-oriented middleware.



Unit-4
Transport and Session Layer Protocols

AMQP

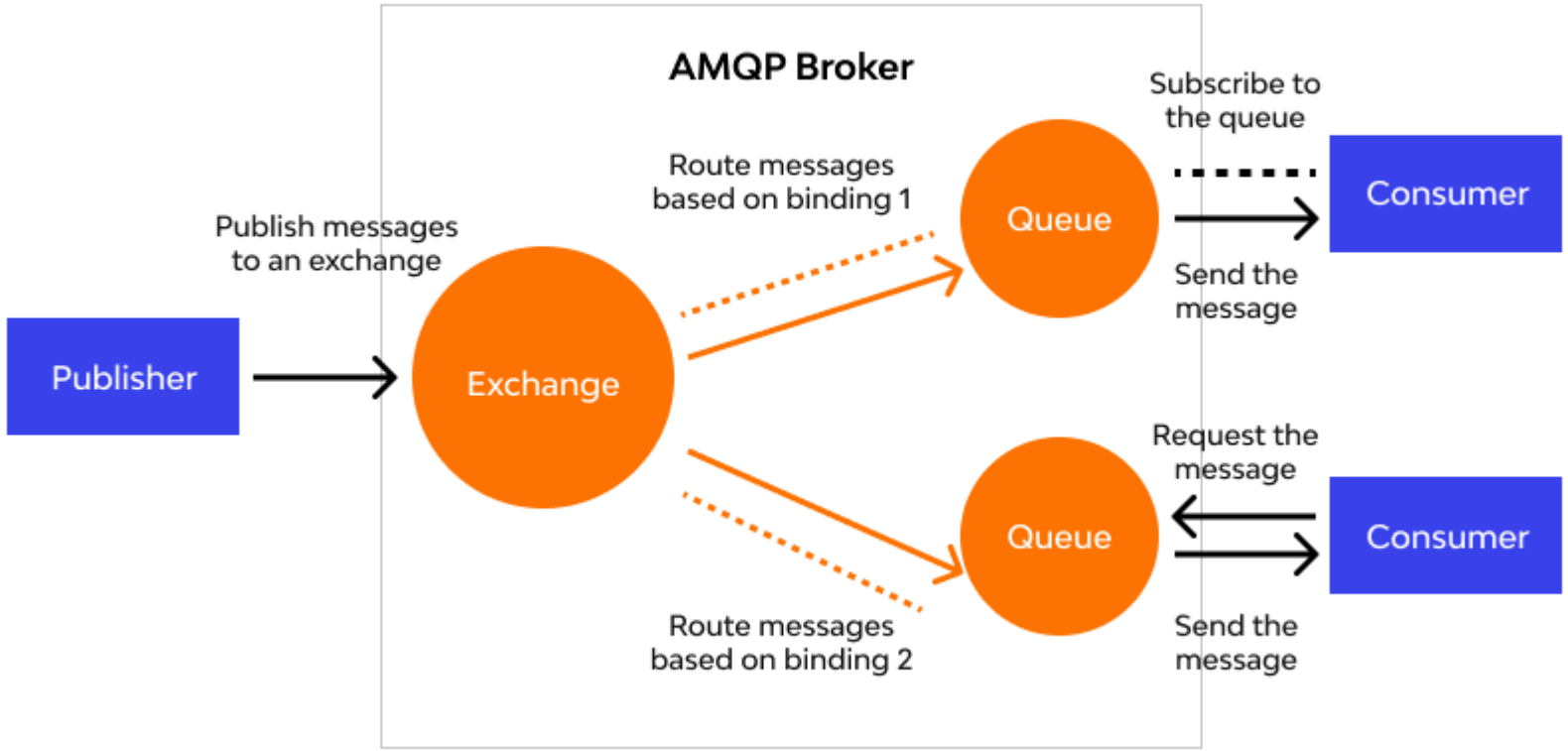
Concepts:

- **Message-Oriented:** AMQP focuses on the exchange of messages between applications.
- **Asynchronous Communication:** It enables decoupled communication where senders and receivers don't need to be active simultaneously.
- **Flexible Routing:** AMQP provides various routing mechanisms, allowing messages to be delivered to specific queues or exchanged based on rules.

Unit-4
Transport and Session Layer Protocols

AMQP

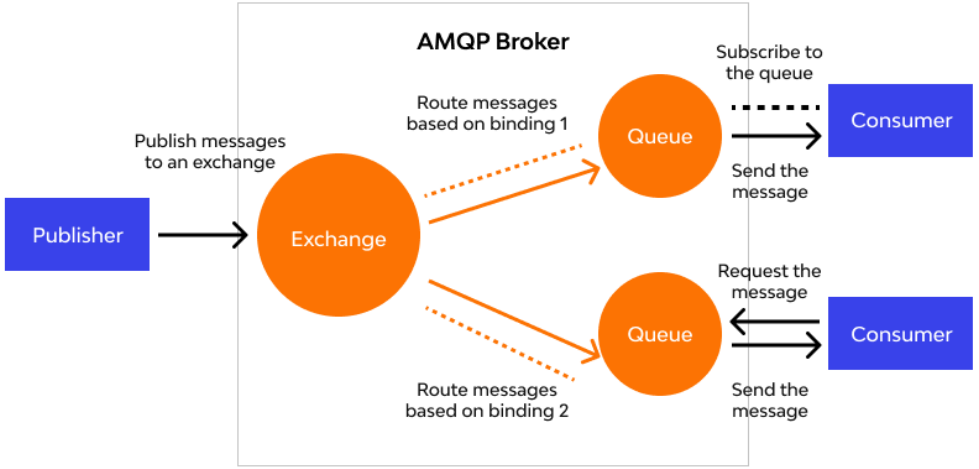
AMQP-Architecture



Unit-4
Transport and Session Layer Protocols

AMQP

AMQP-Architecture



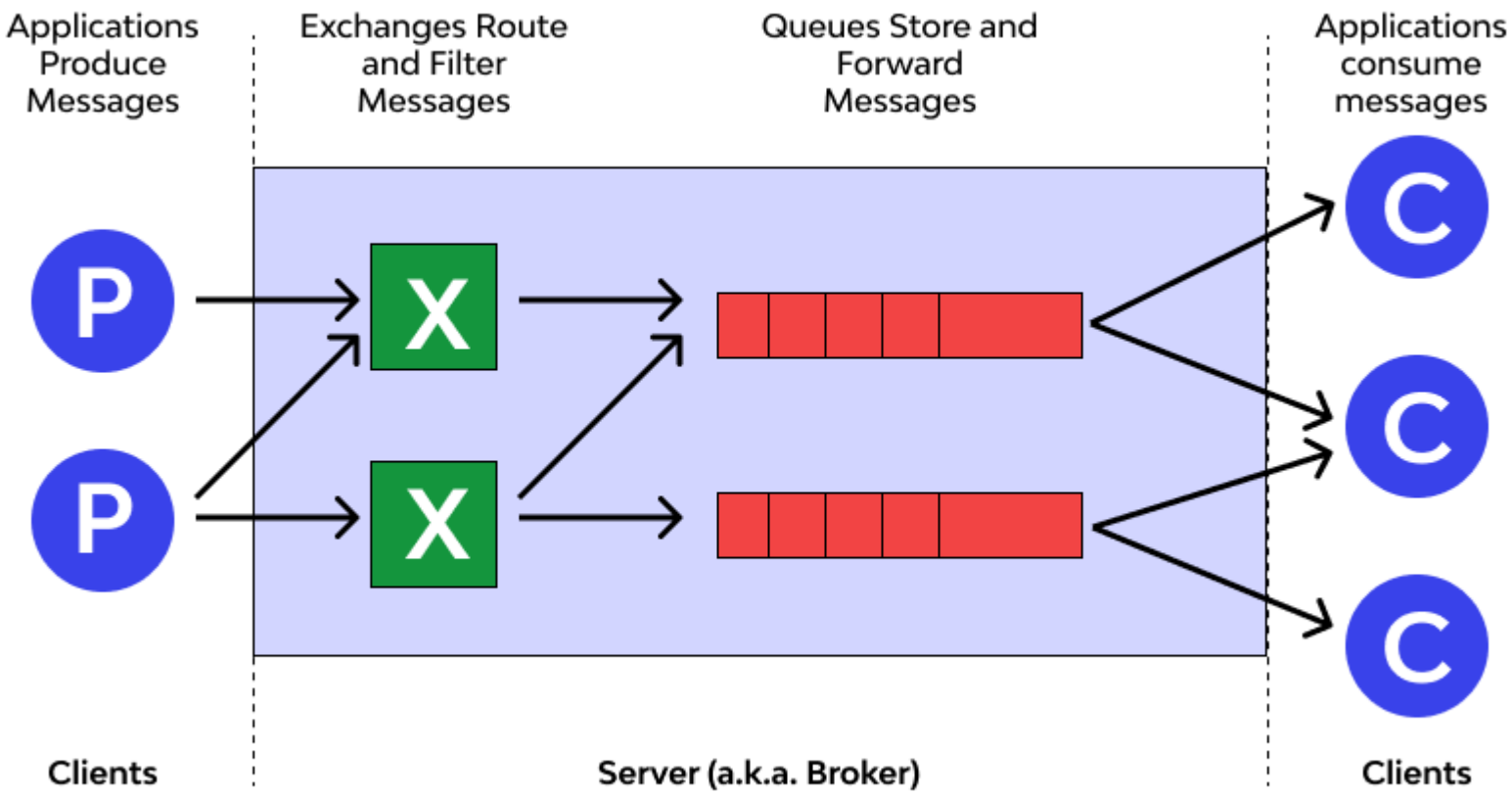
Popular AMQP Brokers:

- **RabbitMQ:** A widely used open-source message broker.
- **Apache ActiveMQ:** Another popular open-source message broker with a wide range of features.
- **Apache Kafka:** A high-throughput, distributed streaming platform that also supports AMQP.

Unit-4
Transport and Session Layer Protocols

AMQP

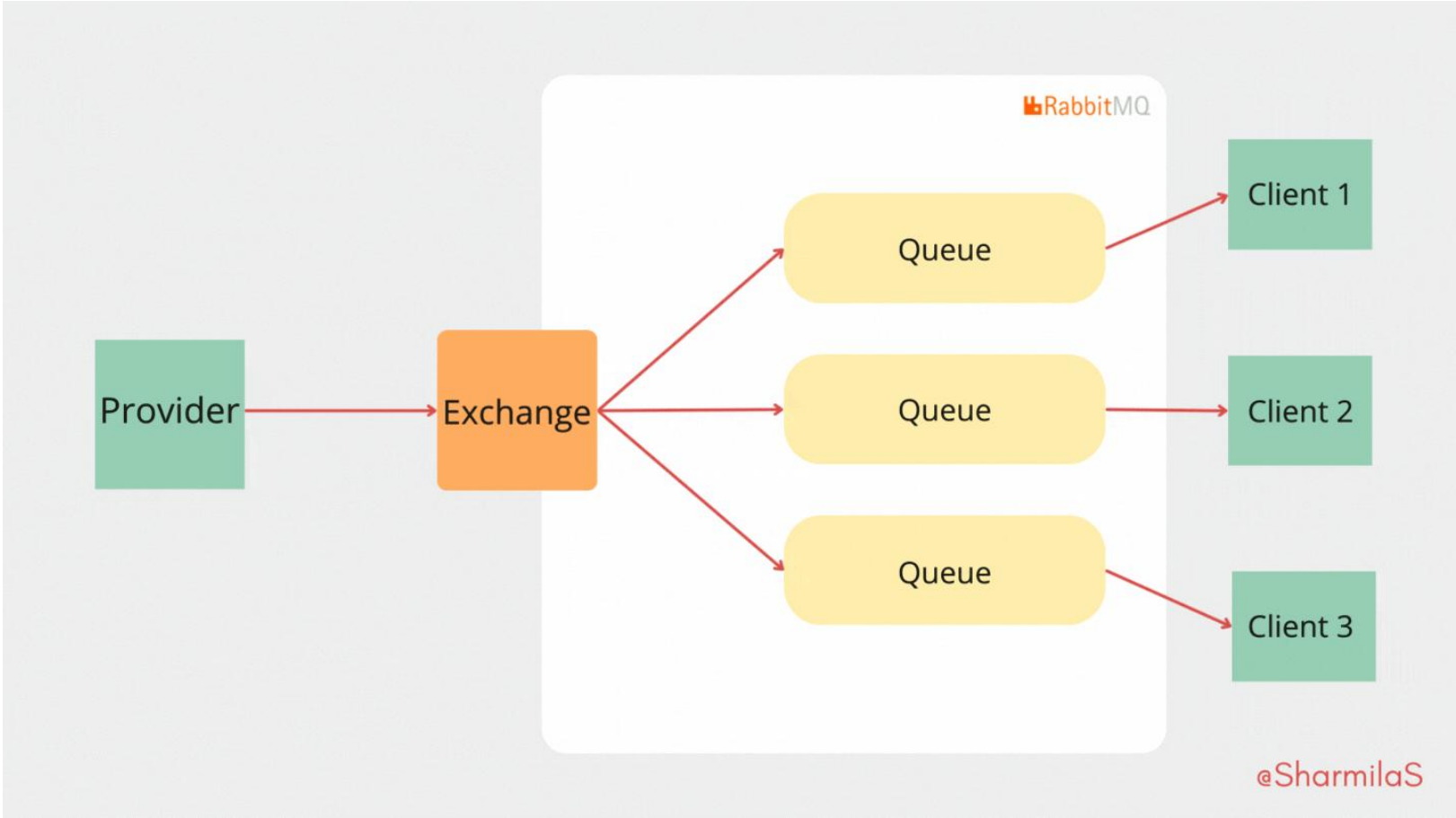
AMQP-Architecture



Unit-4
Transport and Session Layer Protocols

AMQP

AMQP-Working



Unit-4
Transport and Session Layer Protocols

AMQP

How AMQP Works:

1.Producers: Applications that send messages to the message broker.
2.Consumers: Applications that receive messages from the message broker.
3.Message Broker: A central server that acts as an intermediary, handling message routing, queuing, and delivery.

Unit-4
Transport and Session Layer Protocols

AMQP

AMQP Features:

- Reliability: Ensures message delivery with features like message acknowledgments and persistent queues.
- Flexibility: Supports various messaging patterns, including point-to-point, publish-subscribe, and request-reply.
- Scalability: Can handle high message volumes and support a large number of concurrent connections.
- Interoperability: Allows for communication between applications written in different languages and running on different platforms.

Unit-4
Transport and Session Layer Protocols

AMQP

AMQP-Applications

- Microservices Architecture: Enables communication and data exchange between microservices.
- Event-Driven Systems: Facilitates event-driven architectures where applications react to events triggered by other systems.
- Data Integration: Used to integrate applications and systems that use different technologies.
- Real-time Processing: Enables real-time data processing and stream processing applications.

Unit-4
Transport and Session Layer Protocols

AMQP

Pros:

- **Reliable:** Ensures message delivery.
- **Flexible:** Supports various messaging patterns.
- **Scalable:** Handles high message volumes.
- **Interoperable:** Works across different systems.

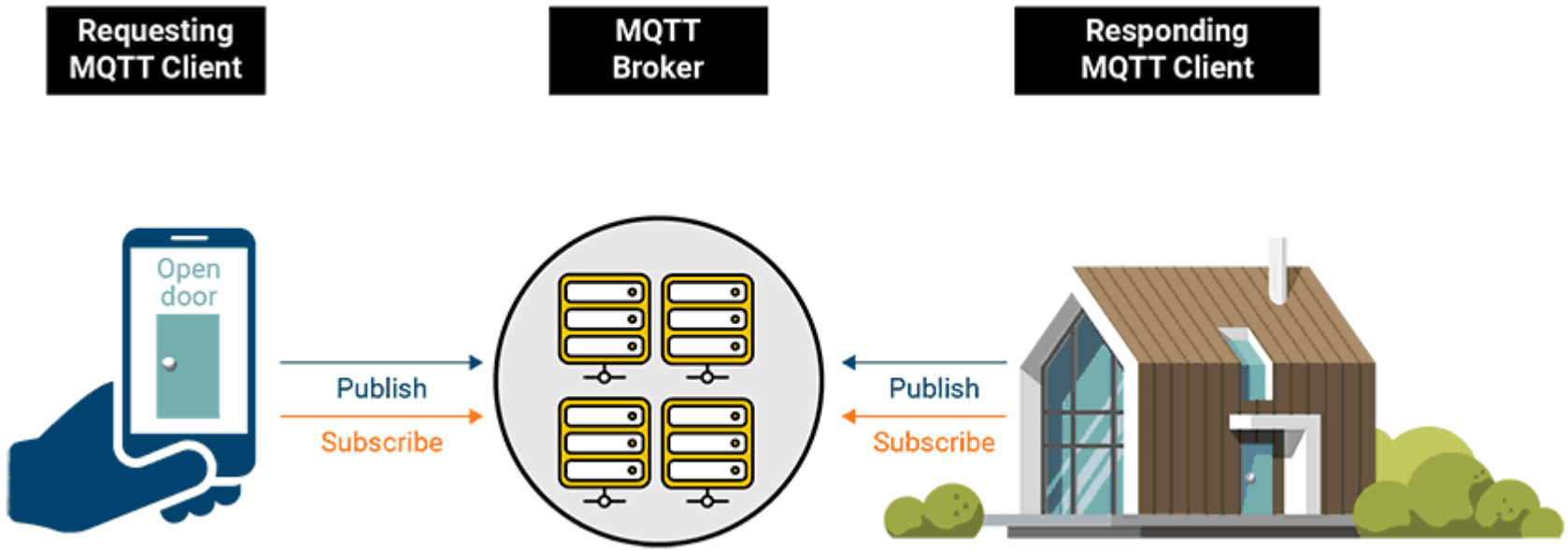
Cons:

- Complex: Can be harder to learn and implement.
- Overhead: May introduce some performance overhead.

Unit-4
Transport and Session Layer Protocols

MQTT

MQTT (Message Queuing Telemetry Transport) is a lightweight messaging protocol specifically designed for constrained environments like the Internet of Things (IoT).



Unit-4
Transport and Session Layer Protocols

MQTT

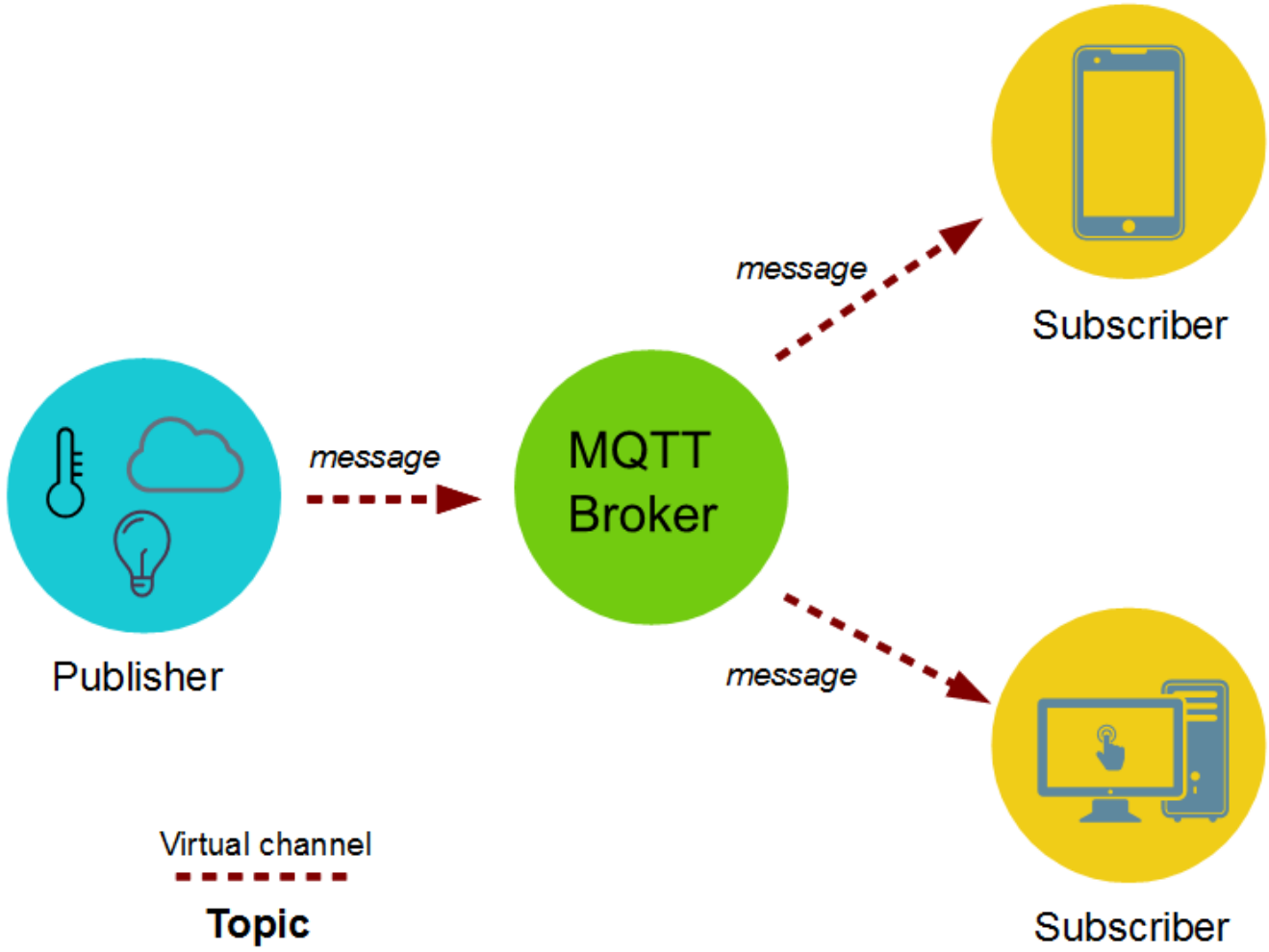
Key Characteristics:

- Lightweight: Minimal overhead, ideal for devices with limited resources.
- Publish-Subscribe: Operates on a publish-subscribe model where devices publish messages to topics and other devices subscribe to those topics to receive1 messages.
- Efficient: Optimized for low bandwidth and high latency networks.
- Simple: Easy to implement and use.

Unit-4
Transport and Session Layer Protocols

MQTT

MQTT-Architecture



Unit-4
Transport and Session Layer Protocols

MQTT

MQTT-Working



Unit-4
Transport and Session Layer Protocols

MQTT

MQTT Client:

MQTT clients can be categorized into two main roles: publishers and subscribers.

•Publishers:

•These are MQTT clients that send or "publish" messages to the MQTT broker. Messages published by publishers are typically associated with specific topics. Topics are like message channels, and messages are sent to a particular topic.

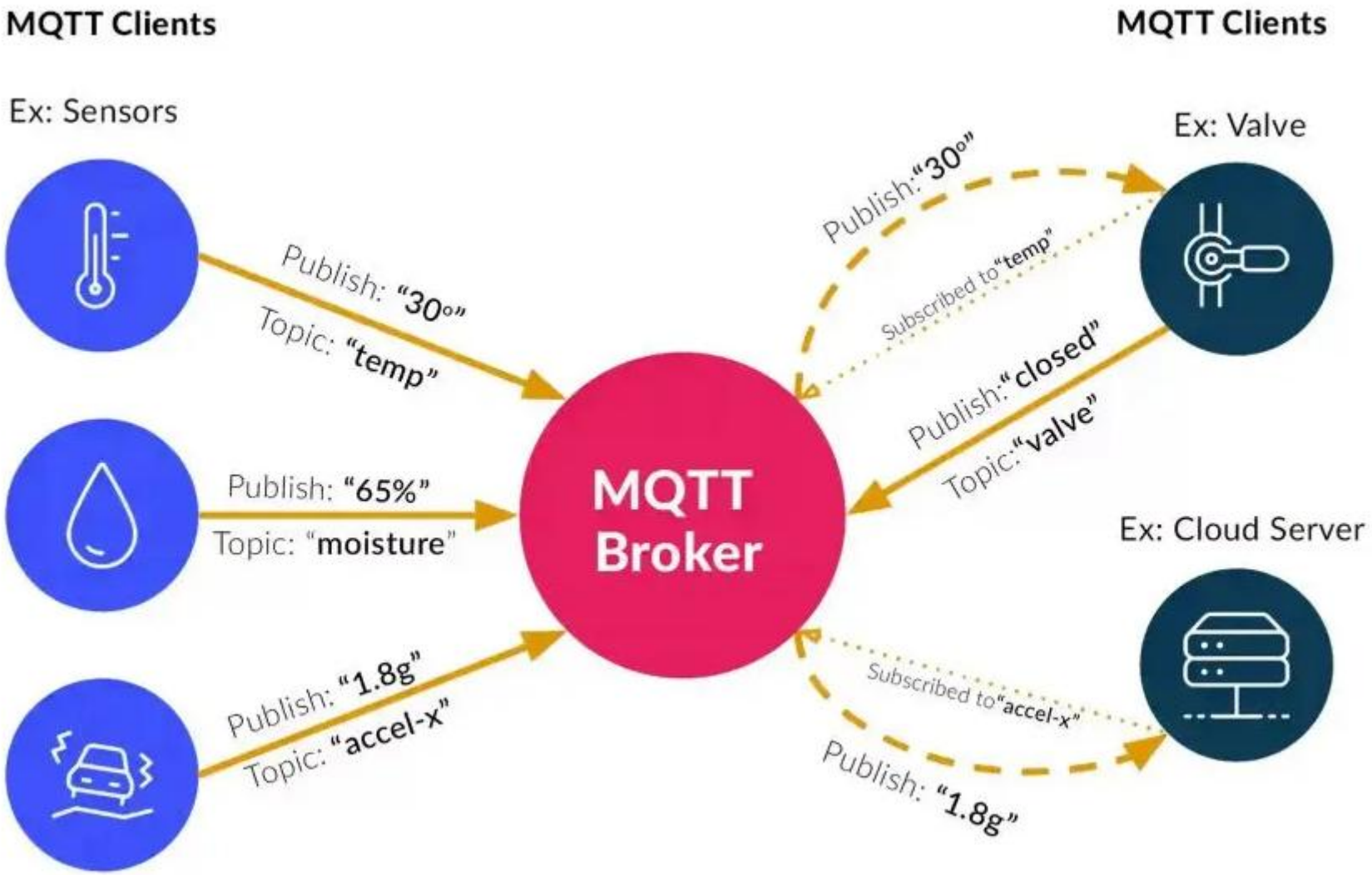
•Subscribers:

•These are MQTT clients that receive or "subscribe" to messages from the MQTT broker. Subscribers express their interest in specific topics, and the broker sends messages published to those topics to the subscribing clients.

Unit-4
Transport and Session Layer Protocols

MQTT

MQTT Client-Example



Unit-4
Transport and Session Layer Protocols

MQTT

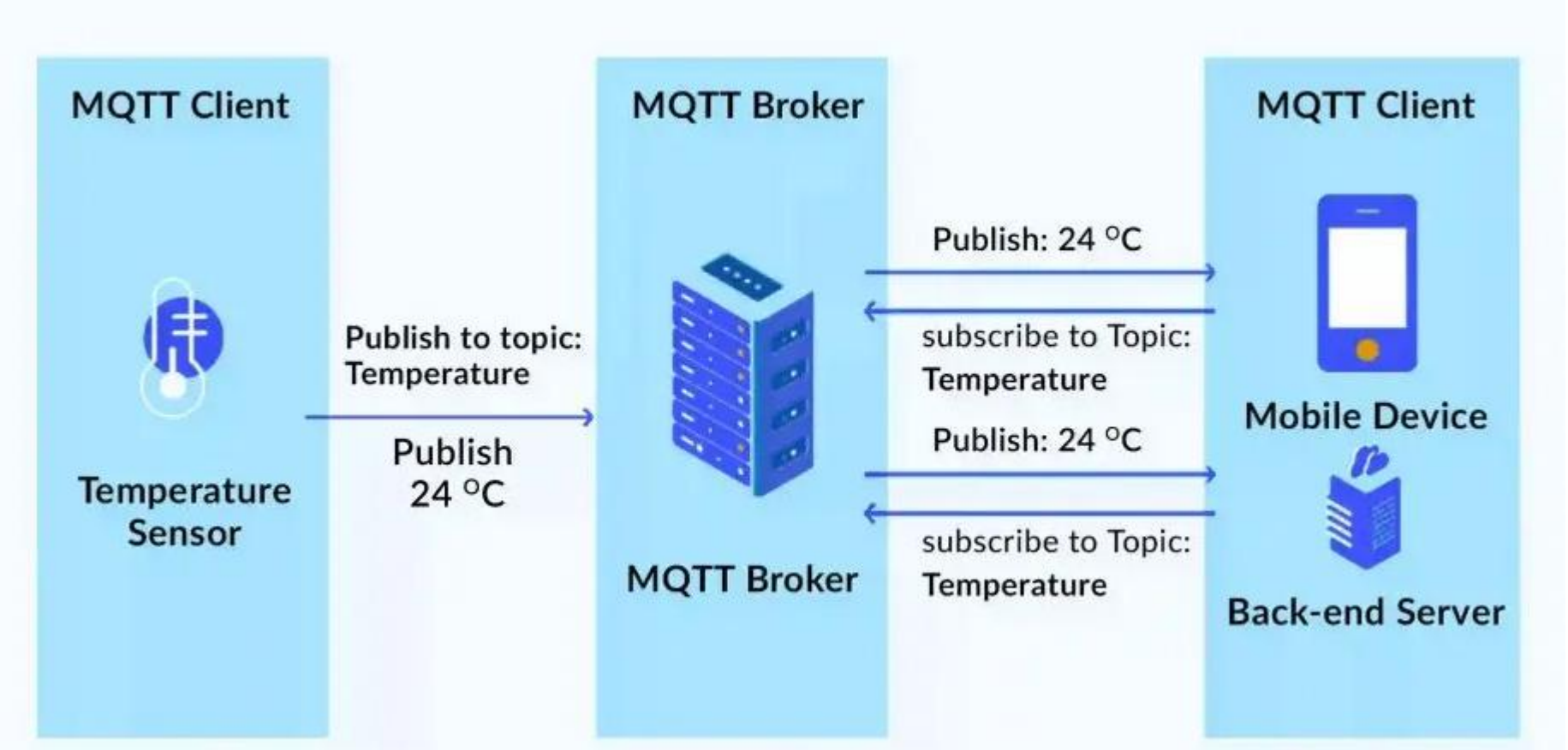
MQTT Broker

- An MQTT broker is a server that receives all messages from the clients and then routes these messages to the appropriate destination clients.
- As a central hub in the MQTT protocol, the broker manages message queues, topic subscriptions, and every MQTT connection, ensuring seamless device communication.
- It also handles the authentication and authorization of clients, ensuring secure and efficient communication in an MQTT network.

Unit-4
Transport and Session Layer Protocols

MQTT

MQTT Broker



Unit-4
Transport and Session Layer Protocols

MQTT

MQTT Topic

- MQTT Topic is a UTF-8 string that brokers use for message filtering.
- Topics are represented as hierarchical strings separated by forward slashes. Clients publish messages to specific topics.
- Subscribers register interest in topics and receive published messages matching those topics.
- The broker handles routing messages from publishers to the appropriate subscribed clients based on topic filtering.
- Example of Topic
  My_home/first_floor/dining_room/temperature

Unit-4
Transport and Session Layer Protocols

MQTT

MQTT-Working



Introduction of Message Queue Telemetry Transport Protocol (MQTT)

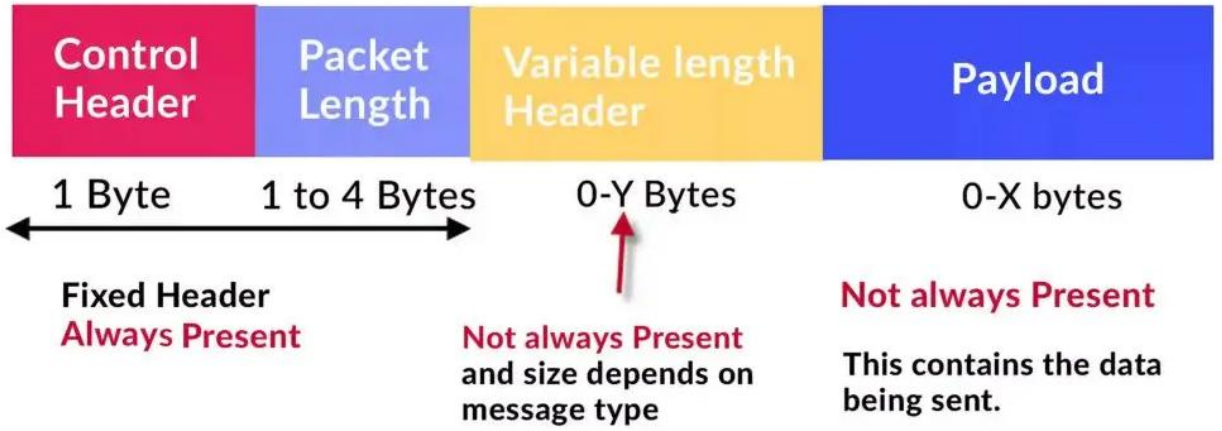
How MQTT Works:

1.Devices connect to an MQTT broker: The broker acts as a central hub for message routing.
2.Devices publish messages to specific topics: For example, a temperature sensor might publish temperature readings to the topic "home/temperature."
3.Devices subscribe to topics of interest: A thermostat might subscribe to the "home/temperature" topic to receive temperature updates.
4.The broker delivers messages to subscribed devices: When a message is published to a topic, the broker delivers it to all subscribed devices.

Unit-4
Transport and Session Layer Protocols

MQTT

MQTT-Message Structure



MQTT messages are made up of three parts:

- Fixed header
- Variable header
- Payload

The fixed header contains information about the type of message and the quality of service level.

The variable header contains information that is specific to each message type.

The MQTT message consists of a payload containing the actual message data published by a client. MQTT protocol limits the maximum payload size to 256 MB per message.

Unit-4
Transport and Session Layer Protocols

MQTT

Advantages of MQTT

•Lightweight protocol that is quick to create and allows for efficient data transport

•Minimal data packet usage, resulting in low network usage

•Effective data dispersion

•The effective use of remote sensing and control

•Prompt and effective message delivery

•Minimises power consumption, which is beneficial for the linked devices, and maximises network capacity.

•Data transmission is quick, efficient, and lightweight because MQTT messages have small code footprint. These control messages have a fixed header of size 2 bytes and payload message up to size 256 megabytes.

Unit-4
Transport and Session Layer Protocols

MQTT

Disadvantages of MQTT

•When compared to Constrained Application Protocol (CoAP), MQTT has slower send cycles.

•Resource discovery in MQTT is based on flexible topic subscription, while resource discovery in CoAP is based on a reliable system.

•MQTT lacks encryption. Rather, security encryption is accomplished by TLS/SSL (Transport Layer Security/Secure Sockets Layer).

•Building an internationally scalable MQTT network is challenging.

Unit-4
Transport and Session Layer Protocols

MQTT

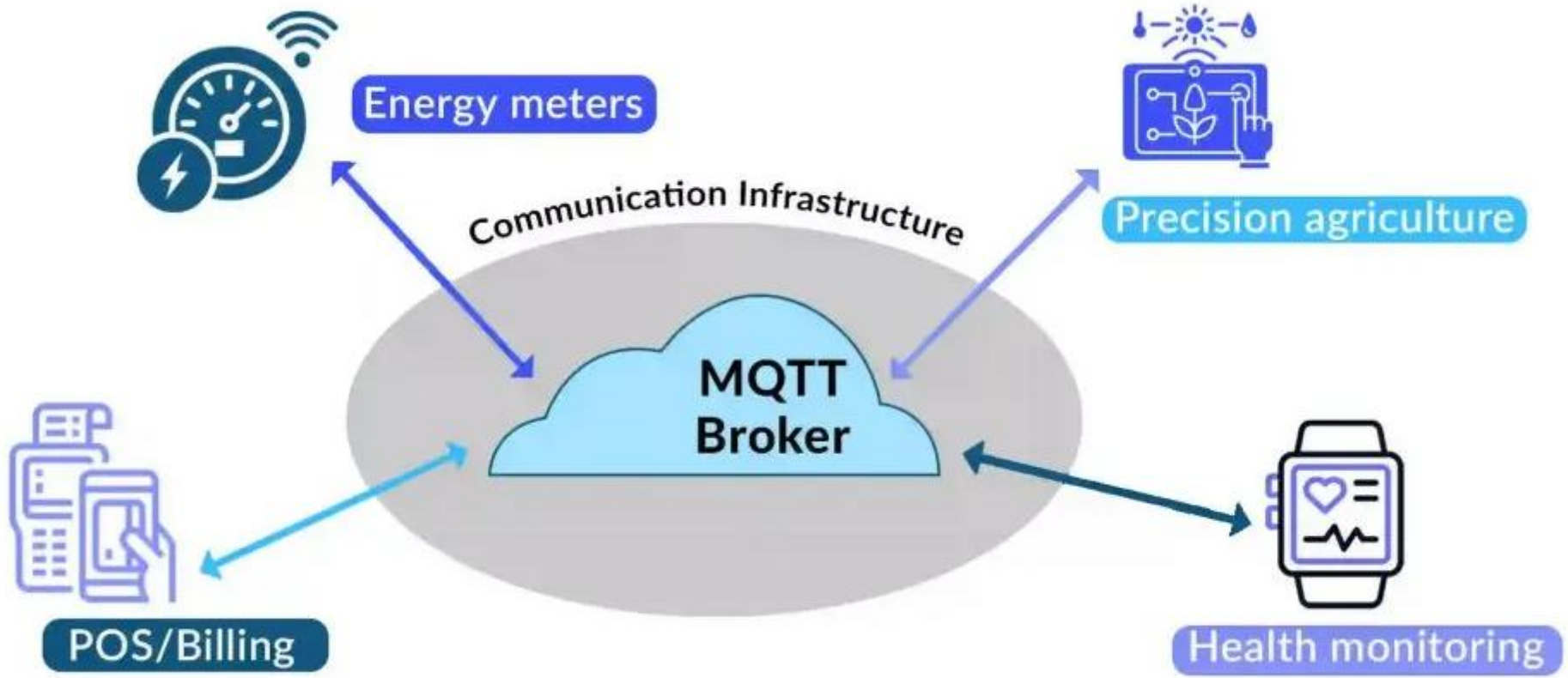
Applications:

- IoT: Connecting sensors, actuators, and other devices in smart homes, industrial automation, and environmental monitoring.
- M2M Communication: Enabling communication between machines in various industries.
- Telematics: Tracking vehicles and other assets.
- Remote Monitoring: Monitoring equipment and systems remotely.

Unit-4
Transport and Session Layer Protocols

MQTT

MQTT Application-Example



Unit-4
Transport and Session Layer Protocols

Comparison of HTTP, CoAP, XMPP, AMQP and MQTT

Protocol	Primary Use Case	Transport Layer	Overhead	Security	Latency	Scalability	QoS	Resource Suitability
HTTP	Web browsing, APIs	TCP	High	HTTPS (TLS/SSL)	High	Moderate	None	Not suitable for constrained devices
CoAP	IoT communication	UDP (primarily)	Low	DTLS	Very low	High	Basic retransmit	Designed for constrained devices

Unit-4
Transport and Session Layer Protocols

Comparison of HTTP, CoAP, XMPP, AMQP and MQTT

Protocol	Primary Use Case	Transport Layer	Overhead	Security	Latency	Scalability	QoS	Resource Suitability
XMPP	Real-time messaging	TCP	Moderate	TLS/SSL	Moderate	High	None	Not optimized for constrained devices
AMQP	Enterprise message queuing	TCP	Moderate to High	TLS/SSL	Low	Very high	Advanced	Requires more resources
MQTT	IoT messaging	TCP	Very low	TLS/SSL	Very low	Very high	3 QoS levels	Designed for constrained devices